Breakout Game — Brief

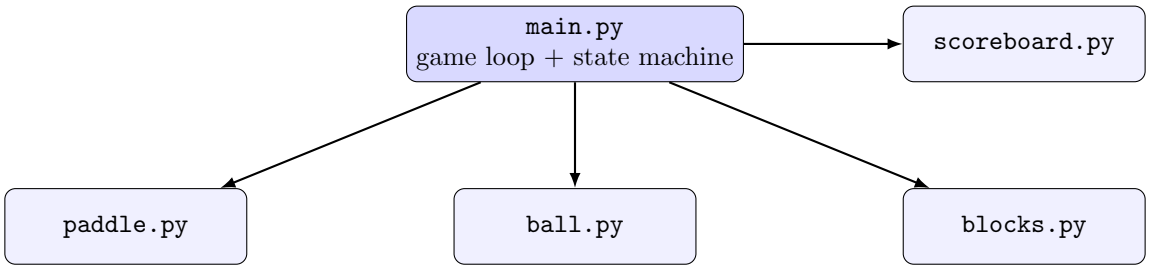
Explainer document (generated for the project owner)

1 Overview

A Breakout clone built with Python's standard-library `turtle` graphics module: no game engine, just a manual game loop (`wn.tracer(0)` + a per-frame `wn.update()`) moving a paddle, a ball, and a wall of blocks. Features: three lives, a difficulty ramp that speeds the ball up as rallies continue, flash-and-particle feedback when a block breaks, start/game-over/win screens, and a high score persisted to `highestScore.txt`.

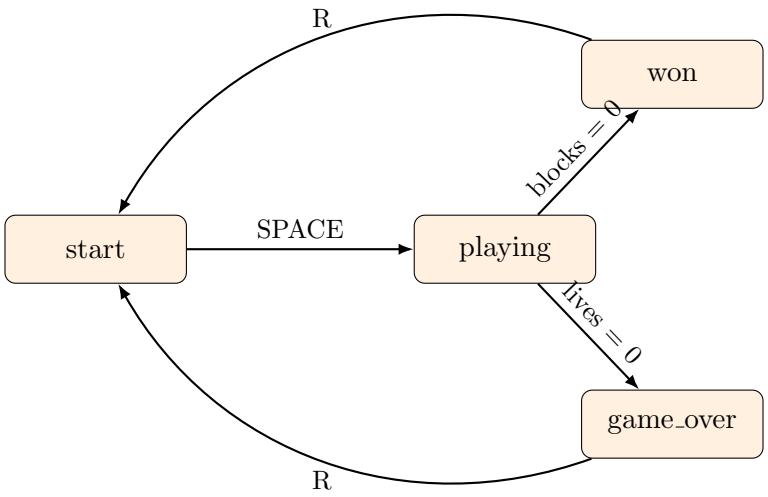
2 Module architecture

Five files, one responsibility each; `main.py` is the only one that wires the others together.



- `main.py`: screen setup, the start/playing/game_over/won state machine, and the frame loop that drives every other module.
- `paddle.py`: `Paddle(Turtle)` — arrow-key movement, clamped to stay on screen.
- `ball.py`: `Ball(Turtle)` — velocity (`dx`, `dy`), all collision checks, the difficulty ramp, and two distinct resets (after a miss vs. a full restart).
- `blocks.py`: `Block` — procedural wall layout, ball-to-block collision, break effects (flash + particles).
- `scoreboard.py`: `Scoreboard(Turtle)` — score/lives display and high-score file persistence.

3 State machine



Physics and collisions only run in `playing`; other states just wait for a keypress, though queued particle bursts keep animating regardless of state so effects don't freeze mid-air.

4 Key mechanics

- **Bouncing**: axis-aligned reflection — hitting a vertical wall flips `dx`, a horizontal wall or the paddle flips `dy`. The paddle only registers a hit when the ball is below $y = -240$ and within its x-range, so it can't catch the ball from an unrealistic angle.
- **Difficulty ramp**: every 4th paddle return multiplies the ball's speed by 1.08 (both axes, direction preserved), capped at 2.6 so it never becomes unplayable. A miss keeps the current speed; only a full restart resets it to the base 1.05.
- **Block layout**: blocks are packed left to right with random widths (3–5 units), wrapping to a new row when one would run off the left edge, and stopping once vertical space runs out. Each row gets one fixed color, so the wall reads as color bands.
- **Collision detection**: a fixed-radius distance check (`block.distance(ball) < 35`) stands in for real hitbox math — cheap, and visually good enough, though it treats every block width the same.
- **Break effects**: a broken block flashes white for 3 frames while 5 small particles fly outward from its position for 8 frames, before both disappear, instead of the block just vanishing.
- **Lives and persistence**: 3 starting lives; a miss respawns the ball at center without resetting difficulty. The high score is read from `highestScore.txt` on startup and rewritten whenever a game ends.

5 Known trade-offs

- Fixed-radius block collision does not account for varying block width.

- The main loop checks ball-block collision twice per frame, likely to reduce ball tunneling through blocks at higher speeds, at the cost of running the check twice as often as usual.
- The high-score file is read at module-import time rather than inside an explicit loader, which works here but would not survive being imported twice.

6 Glossary

turtle
Python's standard-library graphics module.

game loop
Repeatedly moving objects, checking collisions, and redrawing, many times per second.

state machine
A system that is always in exactly one named state, moving between states on specific events.